

How interfaces handle multiple inheritance i java

In Java, a class **cannot extend multiple classes**, but it **can implement multiple interfaces**. This is how Java supports a form of multiple inheritance.

Java

```
interface Printer {
    void print();
}

interface Scanner {
    void scan();
}

class MultiFunctionMachine implements Printer, Scanner {

    @Override
    public void print() {
        System.out.println("Printing...");
    }

    @Override
    public void scan() {
        System.out.println("Scanning...");
    }
}
```

Here, `MultiFunctionMachine` inherits the required behaviors from both `Printer` and `Scanner`.

Why Java uses interfaces for multiple inheritance

Java does not allow this:

Java

```
class C extends A, B {    // Invalid Java
}
```

because multiple class inheritance can create ambiguity, especially the **diamond problem**.

For example:



If both **B** and **C** provide different versions of the same method inherited from **A**, Java would need to decide which one **D** should use.

Interfaces avoid much of this problem because traditionally they specify behavior rather than storing inherited object state.

Multiple interfaces

A class can implement as many interfaces as needed:

Java

```
interface Flyable {
    void fly();
}

interface Swimmable {
    void swim();
}

interface Walkable {
    void walk();
}

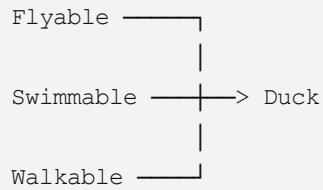
class Duck implements Flyable, Swimmable, Walkable {

    public void fly() {
        System.out.println("Duck flies");
    }

    public void swim() {
        System.out.println("Duck swims");
    }

    public void walk() {
        System.out.println("Duck walks");
    }
}
```

So conceptually:



What about default methods?

Since Java 8, interfaces can contain default methods.

Java

```
interface A {
    default void show() {
        System.out.println("A");
    }
}

interface B {
    default void show() {
        System.out.println("B");
    }
}
```

If a class implements both:

Java

```
class C implements A, B {
}
```

Java gives a compile-time error because it does not know which `show()` method to use.

You must resolve the conflict explicitly:

Java

```
class C implements A, B {

    @Override
    public void show() {
        System.out.println("C");
    }
}
```

Or you can explicitly call one interface's implementation:

Java

```
class C implements A, B {  
  
    @Override  
    public void show() {  
        A.super.show();  
    }  
}
```

Key idea

Java Multiple Inheritance

Classes:

A + B

↓

Class C

 Not allowed

Interfaces:

Interface A

\

→ Class C

/

Interface B

 Allowed

So, **Java handles multiple inheritance through interfaces**, and when two interfaces provide conflicting `default` methods, the implementing class must explicitly resolve the conflict.

can inner classes handle multiple inheritance problem

Yes, but only indirectly.

An **inner class does not give Java true multiple inheritance**. A class still cannot extend two classes:

Java

```
class C extends A, B { } //  Not allowed
```

But you can use inner classes to **combine behavior from multiple parent classes through composition**.

For example:

Java

```
class Car {
    void drive() {
        System.out.println("Driving");
    }
}

class Boat {
    void sail() {
        System.out.println("Sailing");
    }
}

class AmphibiousVehicle {

    class CarPart extends Car {
    }

    class BoatPart extends Boat {
    }

    private CarPart car = new CarPart();
    private BoatPart boat = new BoatPart();

    void drive() {
        car.drive();
    }

    void sail() {
        boat.sail();
    }
}
```

Usage:

Java

```
public class Main {
    public static void main(String[] args) {

        AmphibiousVehicle vehicle = new AmphibiousVehicle();

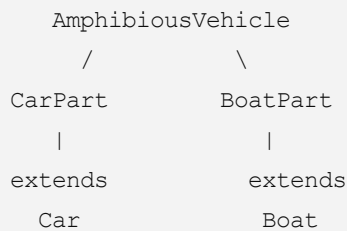
        vehicle.drive();
        vehicle.sail();
    }
}
```

```
}  
}
```

Output:

```
Driving  
Sailing
```

Here:



So `AmphibiousVehicle` can expose functionality from both `Car` and `Boat`, but it **does not itself inherit from both**.

This technique is essentially **composition/delegation using inner classes**.

For most Java designs, though, interfaces are cleaner:

Java

```
interface Drivable {  
    void drive();  
}  
  
interface Sailable {  
    void sail();  
}  
  
class AmphibiousVehicle implements Drivable, Sailable {  
  
    public void drive() {  
        System.out.println("Driving");  
    }  
  
    public void sail() {  
        System.out.println("Sailing");  
    }  
}
```

So the distinction is:

Technique	Multiple inheritance?	Java support
<code>class C extends A, B</code>	True class multiple inheritance	✗
Multiple interfaces	Multiple inheritance of type/behavior	✓
Inner classes	Simulates combining multiple class behaviors	✓
Composition/delegation	Combines objects instead of inheriting	✓

Inner classes can therefore help **work around the need for multiple class inheritance**, but they do not actually enable it.

Lets suppose A Tarvel base class first extends Car and Car will be base class Boat and than AmphibiousVehicle is used, Demonstrate code.

Yes. You can build a chain like this:

```

Travel
  ↑
Car
  ↑
Boat
  ↑
AmphibiousVehicle

```

But note: this is **multilevel inheritance**, not multiple inheritance.

Java

```

class Travel {
    void move() {
        System.out.println("Traveling...");
    }
}

class Car extends Travel {
    void drive() {
        System.out.println("Driving on road...");
    }
}

```

```

}

class Boat extends Car {
    void sail() {
        System.out.println("Sailing on water...");
    }
}

class AmphibiousVehicle extends Boat {
    void amphibiousMode() {
        System.out.println("Amphibious vehicle can travel on road and water.");
    }
}

```

Now use it:

Java

```

public class Main {
    public static void main(String[] args) {

        AmphibiousVehicle vehicle = new AmphibiousVehicle();

        vehicle.move();           // inherited from Travel
        vehicle.drive();           // inherited from Car
        vehicle.sail();            // inherited from Boat
        vehicle.amphibiousMode(); // own method
    }
}

```

Output:

```

Traveling...
Driving on road...
Sailing on water...
Amphibious vehicle can travel on road and water.

```

The inheritance path is:

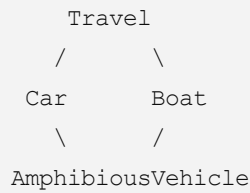
```

Travel
|
v
Car
|
v
Boat
|
v
AmphibiousVehicle

```


So `AmphibiousVehicle` receives all accessible methods from its ancestors.

However, conceptually, making `Boat extends Car` is usually a poor design because a **Boat is not a Car**. A better real-world design would be:



That diagram represents the multiple-inheritance problem, and Java would normally solve it using **interfaces or composition**, not `Boat extends Car`.

is this wrong i got functionality of both i don't need to override methods

It is not *syntactically* wrong. Your code works, and `AmphibiousVehicle` gets both `drive()` and `sail()` without overriding anything.

The issue is mainly **object-oriented design**.

With:

Java

```
class Travel {
    void move() { }
}

class Car extends Travel {
    void drive() { }
}

class Boat extends Car {
    void sail() { }
}

class AmphibiousVehicle extends Boat {
}
```

`AmphibiousVehicle` inherits:

```
move()   from Travel
drive()  from Car
sail()   from Boat
```

So this is valid:

Java

```
public class Main {
    public static void main(String[] args) {
        AmphibiousVehicle a = new AmphibiousVehicle();

        a.move();
        a.drive();
        a.sail();
    }
}
```

You are correct that **you do not need to override methods** if the inherited implementations already do exactly what you want.

The concern is this relationship:

Java

```
class Boat extends Car
```

Inheritance means **"is-a"**.

So this declaration says:

```
Boat IS-A Car
```

which is usually logically incorrect.

It also causes consequences like this:

Java

```
Car c = new Boat(); // Java allows this!
```

because you told Java that every `Boat` is a kind of `Car`.

That is why your solution works technically, but it is not really solving multiple inheritance. It is creating **multilevel inheritance**:

```
Travel
  ↑
Car
  ↑
Boat
  ↑
AmphibiousVehicle
```

A true multiple-inheritance situation would be:

```
      Travel
     /    \
    Car    Boat
     \    /
  AmphibiousVehicle
```

You want `AmphibiousVehicle` to receive functionality from both `Car` and `Boat` independently.

Java does not allow:

Java

```
class AmphibiousVehicle extends Car, Boat { } // ❌
```

So the important distinction is:

Your approach: works without overriding, but forces `Boat` to be a subclass of `Car`.

Multiple inheritance: `Car` and `Boat` are separate siblings, and `AmphibiousVehicle` inherits from both.

So yes, your code can absolutely reuse both methods without overriding. It is only considered a poor model when `Boat is-a Car` does not make sense.

Well I dont agree with you a Boat can be simple boat but if it has an engine we can inherit it from car

You can model it that way, but the key question is not whether both have an engine. The question is whether a `Boat` is truly a specialized kind of `Car`.

For example:

Java

```
class Car {
    void startEngine() {
        System.out.println("Engine started");
    }

    void drive() {
        System.out.println("Driving on road");
    }
}

class Boat extends Car {
    void sail() {
        System.out.println("Moving on water");
    }
}
```

Now Java considers every `Boat` to be a `Car`, so this becomes valid:

Java

```
Car c = new Boat();

c.startEngine(); // makes sense
c.drive();       // logically strange for a normal boat
```

That is the real issue. You inherited not only the engine functionality, but also the whole identity and behavior of `Car`.

A better model would usually extract the common capability:

Java

```
class Engine {
    void start() {
        System.out.println("Engine started");
    }
}

class Car {
    Engine engine = new Engine();

    void drive() {
        engine.start();
        System.out.println("Driving on road");
    }
}

class Boat {
```

```

Engine engine = new Engine();

void sail() {
    engine.start();
    System.out.println("Moving on water");
}
}

```

Then an amphibious vehicle can combine both behaviors cleanly.

So your idea is valid as a programming shortcut, and Java will allow it. But inheriting `Boat` from `Car` specifically because both have engines mixes up two relationships:

```

Inheritance:  Boat IS-A Car
Composition: Boat HAS-AN Engine

```

For an engine, `HAS-A` is normally the more accurate relationship.

Branched from [Explain Java Interfaces](#)

courses.json
File

grades.json
File

students.json
File

Create a Class of DataSheet which calculate CGPA of a Student by using ArrayList of Courses and Grades. Assign total marks to all courses of a particular randomly

Using your uploaded data structure, each course contains `courseid`, `code`, `title`, `crhr`, and `semester`; for example, *Calculus and Analytical Geometry* has 3 credit hours in semester 1. `courses` Your grade table maps marks ranges to GPA values, such as 95–100 → A+ → 4.0. `grades` The student data contains registration numbers and names. `students`

A good OOP design is:

```

Student
|
v
DataSheet
|
+---- ArrayList<Course>

```

```
|  
+---- ArrayList<Grade>
```

DataSheet can randomly assign marks to every course of one student and then calculate the student's CGPA using credit hours.

Java

```
import java.util.ArrayList;  
import java.util.Random;  
  
class Student {  
    private String regNo;  
    private String name;  
  
    public Student(String regNo, String name) {  
        this.regNo = regNo;  
        this.name = name;  
    }  
  
    public String getRegNo() {  
        return regNo;  
    }  
  
    public String getName() {  
        return name;  
    }  
}
```

Course class

Java

```
class Course {  
    private int courseId;  
    private String code;  
    private String title;  
    private int creditHours;  
    private int semester;  
  
    private int marks;  
  
    public Course(int courseId, String code, String title,  
                  int creditHours, int semester) {  
  
        this.courseId = courseId;  
        this.code = code;  
        this.title = title;  
        this.creditHours = creditHours;  
        this.semester = semester;  
    }  
}
```

```

    }

    public int getCourseId() {
        return courseId;
    }

    public String getCode() {
        return code;
    }

    public String getTitle() {
        return title;
    }

    public int getCreditHours() {
        return creditHours;
    }

    public int getSemester() {
        return semester;
    }

    public int getMarks() {
        return marks;
    }

    public void setMarks(int marks) {
        this.marks = marks;
    }
}

```

Grade class

Java

```

class Grade {
    private int start;
    private int end;
    private String grade;
    private double gpa;

    public Grade(int start, int end, String grade, double gpa) {
        this.start = start;
        this.end = end;
        this.grade = grade;
        this.gpa = gpa;
    }

    public boolean contains(int marks) {
        return marks >= start && marks <= end;
    }
}

```

```

    public String getGrade() {
        return grade;
    }

    public double getGpa() {
        return gpa;
    }
}

```

DataSheet class

This is the main class that contains the `ArrayList<Course>` and `ArrayList<Grade>`.

Java

```

class DataSheet {

    private Student student;

    private ArrayList<Course> courses;
    private ArrayList<Grade> grades;

    public DataSheet(Student student) {
        this.student = student;

        courses = new ArrayList<>();
        grades = new ArrayList<>();
    }

    public void addCourse(Course course) {
        courses.add(course);
    }

    public void addGrade(Grade grade) {
        grades.add(grade);
    }

    // Assign random marks from 0 to 100
    public void assignRandomMarks() {

        Random random = new Random();

        for (Course course : courses) {
            int marks = random.nextInt(101);

            course.setMarks(marks);
        }
    }

    public Grade findGrade(int marks) {

```



```

        for (Grade grade : grades) {

            if (grade.contains(marks)) {
                return grade;
            }
        }

        return null;
    }

    public double calculateCGPA() {

        double totalQualityPoints = 0;
        int totalCreditHours = 0;

        for (Course course : courses) {

            Grade grade = findGrade(course.getMarks());

            if (grade != null) {

                totalQualityPoints +=
                    grade.getGpa() * course.getCreditHours();

                totalCreditHours += course.getCreditHours();
            }
        }

        if (totalCreditHours == 0) {
            return 0;
        }

        return totalQualityPoints / totalCreditHours;
    }

    public void printDataSheet() {

        System.out.println("\nStudent: " + student.getName());
        System.out.println("Reg No : " + student.getRegNo());

        System.out.println(
            "-----");

        System.out.printf("%-10s %-30s %-5s %-7s %-6s\n",
            "Code",
            "Course",
            "CrHr",
            "Marks",
            "Grade");

        System.out.println(
            "-----");
    }

```

```

        for (Course course : courses) {

            Grade grade = findGrade(course.getMarks());

            System.out.printf("%-10s %-30s %-5d %-7d %-6s%n",
                               course.getCode(),
                               course.getTitle(),
                               course.getCreditHours(),
                               course.getMarks(),
                               grade.getGrade());
        }

        System.out.println(
            "-----");

        System.out.printf("CGPA: %.2f%n", calculateCGPA());
    }
}

```

Main class

Here I am using one student from your uploaded file and some semester-1 courses.

Java

```

public class Main {

    public static void main(String[] args) {

        Student student =
            new Student("1712218", "Aadesh Kumar");

        DataSheet sheet = new DataSheet(student);

        // -----
        // Courses
        // -----

        sheet.addCourse(
            new Course(
                1,
                "CS1102",
                "Calculus and Analytical Geometry",
                3,
                1));

        sheet.addCourse(
            new Course(
                2,
                "CS1111",

```

```

        "English Composition",
        3,
        1));

sheet.addCourse(
    new Course(
        3,
        "CS1114",
        "Introduction to Computer Sciences",
        4,
        1));

sheet.addCourse(
    new Course(
        4,
        "CS1118",
        "Fundamentals of Programming",
        4,
        1));

sheet.addCourse(
    new Course(
        5,
        "CS1206",
        "Islamiat / Pakistan Studies",
        3,
        1));

// -----
// Grade Scale
// -----

sheet.addGrade(new Grade(95, 100, "A+", 4.00));
sheet.addGrade(new Grade(91, 94, "A", 3.75));
sheet.addGrade(new Grade(87, 90, "A-", 3.50));
sheet.addGrade(new Grade(83, 86, "B+", 3.25));
sheet.addGrade(new Grade(79, 82, "B", 3.00));
sheet.addGrade(new Grade(75, 78, "B-", 2.75));
sheet.addGrade(new Grade(71, 74, "C+", 2.50));
sheet.addGrade(new Grade(68, 70, "C", 2.25));
sheet.addGrade(new Grade(65, 67, "C-", 2.00));
sheet.addGrade(new Grade(60, 64, "D", 1.00));
sheet.addGrade(new Grade(0, 59, "F", 0.00));

// Randomly assign marks
sheet.assignRandomMarks();

// Print student's data sheet
sheet.printDataSheet();

```

A possible output would be:

Student: Aadesh Kumar

Reg No : 1712218

Code	Course	CrHr	Marks	Grade
CS1102	Calculus and Analytical Geometry	3	86	B+
CS1111	English Composition	3	92	A
CS1114	Introduction to Computer Sciences	4	78	B-
CS1118	Fundamentals of Programming	4	95	A+
CS1206	Islamiat / Pakistan Studies	3	72	C+

CGPA: 3.10

The important CGPA calculation is **credit-hour weighted**:

Java

```
totalQualityPoints +=  
    grade.getGpa() * course.getCreditHours();
```

and finally:

Java

```
CGPA = totalQualityPoints / totalCreditHours;
```

So a 4-credit-hour course contributes more to the CGPA than a 3-credit-hour course.

Sources

use Lambda functions in calculateCGPA for getting grade rather than to have a separate function findGrades()

Yes. You can remove `findGrade()` completely and use a **lambda inside**

`calculateCGPA()` to locate the matching grade from the `ArrayList<Grade>`.

Your grade data maps mark ranges to grade/GPA values such as `95-100 → A+ → 4.0`.

grades

Use this version:

Java

```
public double calculateCGPA() {

    double totalQualityPoints = 0;
    int totalCreditHours = 0;

    for (Course course : courses) {

        Grade grade = grades.stream()
            .filter(g -> course.getMarks() >= g.getStart()
                && course.getMarks() <= g.getEnd())
            .findFirst()
            .orElse(null);

        if (grade != null) {
            totalQualityPoints +=
                grade.getGpa() * course.getCreditHours();

            totalCreditHours += course.getCreditHours();
        }
    }

    return totalCreditHours == 0
        ? 0
        : totalQualityPoints / totalCreditHours;
}
```

For that to work, add these getters to `Grade`:

Java

```
public int getStart() {
    return start;
}

public int getEnd() {
    return end;
}
```

The important lambda is:

Java

```
.filter(g -> course.getMarks() >= g.getStart()
    && course.getMarks() <= g.getEnd())
```

Here, `g` represents each `Grade` object in the `ArrayList<Grade>`.

So for marks `86`, Java effectively checks:

```
86 against A+ → false
86 against A  → false
86 against A- → false
86 against B+ → true
```

Then:

Java

```
.findFirst()
```

returns the matching `Grade`.

You can make `calculateCGPA()` even more stream/lambda-based:

Java

```
public double calculateCGPA() {

    double totalQualityPoints = courses.stream()
        .mapToDouble(course -> {

            Grade grade = grades.stream()
                .filter(g -> course.getMarks() >= g.getStart()
                    && course.getMarks() <= g.getEnd())
                .findFirst()
                .orElse(null);

            if (grade == null)
                return 0;

            return grade.getGpa() * course.getCreditHours();
        })
        .sum();

    int totalCreditHours = courses.stream()
        .mapToInt(course -> course.getCreditHours())
        .sum();

    return totalCreditHours == 0
        ? 0
        : totalQualityPoints / totalCreditHours;
}
```

I would use the **second version** for demonstrating Java lambdas because it clearly shows:

Java

```
courses.stream()  
    .mapToDouble(course -> ...)  
    .sum();
```

and, inside it:

Java

```
grades.stream()  
    .filter(g -> ...)  
    .findFirst();
```

So you're using lambdas both to **search the grade table** and to **calculate the CGPA**.

Sources

ChatGPT can make mistakes. Check important info.